

LAPORAN PRATIKUM STRUKTUR DATA

PEKAN 9

disusun Oleh:

Habib Al Faruq

2511532010

Dosen Pengampu: Dr. Wahyudi S.T.M.T

Asisten Praktikum: Muhammad Galid Avero



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, penulis dapat menyelesaikan laporan praktikum struktur data mengenai penggunaan Algoritma Sorting. Penulisan laporan ini bertujuan untuk membuat sebuah laporan tentang materi pemrograman sort di Java yang meliputi Merge Sort, Quick Sort, dan Shell Sort

Penyusunan laporan praktikum ini tidak terlepas dari bantuan berbagai pihak. Oleh karena itu, penulis ingin menyampaikan terima kasih kepada dosen pengampu mata kuliah praktikum pemrograman algoritma dan pemrograman, Bapak Dr. Wahyudi. S.T.M.T dan asisten praktikum, Uda Muhammad Galid Averro yang telah memberikan arahan dan masukan dalam proses penyusunan laporan praktikum ini. Penulis berharap laporan praktikum ini dapat memberikan manfaat bagi pembaca dalam mempelajari konsep penggunaan Algoritma Sorting di Java.

DAFTAR ISI

KATA PENGANTAR	1
DAFTAR ISI	2
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang Praktikum	3
1.2 Tujuan Praktikum	3
BAB II PEMBAHASAN.....	5
2.1 Node_2511532010	5
2.2 BTree_2511532010	8
2.3 BtreeDriver_2511532010.....	11
2.4 GraphTransversal_2511532010.....	14
BAB III KESIMPULAN	17
3.1 Ringkasan Hasil Praktikum.....	17
DAFTAR PUSTAKA.....	18
TUGAS	19

BAB I PENDAHULUAN

1.1 Latar Belakang Praktikum

Perkembangan teknologi informasi menuntut pengolahan data yang cepat, efisien, dan terstruktur, terutama ketika sistem harus menangani volume informasi yang terus membesar dan semakin kompleks. Dalam rekayasa perangkat lunak, pemilihan struktur data yang tepat—baik linear maupun non-linear seperti *tree* dan *graph*—serta penggunaan algoritma yang menyertainya (seperti algoritma *sorting* dan *traversal*) sangat menentukan performa dan kecepatan komputasi dari sebuah program. Tanpa pemahaman mendalam mengenai fondasi struktur memori dan metode manipulasi data yang optimal, seorang pengembang akan kesulitan membangun aplikasi yang berskala besar dan responsif. Oleh karena itu, praktikum ini diadakan sebagai jembatan pembuktian antara teori komputasi yang didapatkan di kelas dengan implementasi nyata, sehingga mahasiswa dapat menyimulasikan langsung bagaimana data disusun, saling dihubungkan, dan dikelola di dalam memori komputer.

1.2 Tujuan Praktikum

Tujuan utama dari pelaksanaan praktikum ini adalah melatih kemampuan teknis mahasiswa dalam menerjemahkan logika algoritmik ke dalam baris kode fungsional menggunakan bahasa pemrograman berbasis objek. Melalui kegiatan ini, mahasiswa dituntut untuk mampu secara mandiri merancang, menulis, dan mengeksekusi operasi pengurutan data (*Shell*, *Quick*, *Merge Sort*) serta merakit relasi data kompleks berupa pohon biner (*Binary Tree*) dan jejaring simpul (*Graph*). Selain itu, praktikum ini secara khusus bertujuan untuk mempertajam daya analisis pemecahan masalah (*problem-solving*) mahasiswa melalui penerapan penelusuran data secara mendalam maupun melebar (*DFS*, *BFS*, *InOrder*, *PreOrder*, *PostOrder*) dengan mematuhi aturan penamaan dan struktur *class* yang sistematis.

1.3 Manfaat Praktikum

Manfaat utama yang akan diperoleh mahasiswa dari praktikum ini adalah terbentuknya insting analitis dan kompetensi teknis yang kuat untuk memilih rancangan algoritma dengan kompleksitas ruang dan waktu paling efisien. Dengan berlatih menyelesaikan berbagai studi kasus riil—seperti mengurutkan *playlist* lagu berdasarkan kriteria tertentu atau memetakan keterhubungan antarsimpul—mahasiswa memiliki bekal pengalaman komprehensif yang sangat dibutuhkan oleh industri perangkat lunak profesional. Pada akhirnya, kebiasaan menulis kode (*coding*) yang terstruktur dalam praktikum ini akan menjadi fondasi jangka panjang yang solid bagi mahasiswa untuk mempelajari keilmuan komputer tingkat lanjut, seperti manajemen basis data, algoritma perutean cerdas, hingga rekayasa kecerdasan buatan.

BAB II PEMBAHASAN

2.1 Node_2511532010

```
1 package pekan9_2511532010;
2
3 public class Node_2511532010 {
4     int data;
5     Node_2511532010 left;
6     Node_2511532010 right;
7
8     public Node_2511532010(int data) {
9         this.data = data;
10        left = null;
11        right = null;
12    }
13
14    public void setLeft(Node_2511532010 node) {
15        if (left == null)
16            left = node;
17    }
18
19    public void setRight(Node_2511532010 node) {
20        if (right == null)
21            right = node;
22    }
23
24    public Node_2511532010 getLeft() {
25        return left;
26    }
27
28    public Node_2511532010 getRight() {
29        return right;
30    }
31
32    public int getData() {
33        return data;
34    }
35
36    public void setData(int data) {
37        this.data = data;
38    }
39
40    // Traversal Methods
41    void printPreorder(Node_2511532010 node) {
42        if (node == null) return;
43        System.out.print(node.data + " ");
44        printPreorder(node.left);
45        printPreorder(node.right);
46    }
47
48    void printPostorder(Node_2511532010 node) {
49        if (node == null) return;
50        printPostorder(node.left);
51        printPostorder(node.right);
52        System.out.print(node.data + " ");
53    }
54
55    void printInorder(Node_2511532010 node) {
56        if (node == null) return;
57        printInorder(node.left);
58        System.out.print(node.data + " ");
59        printInorder(node.right);
60    }
61
62    public void print() {
63        this.print("", true);
64    }
65
66    public void print(String prefix_2010, boolean isTail_2010) {
67        if (right != null) {
68            right.print(prefix_2010 + (isTail_2010 ? "| " : " "), false);
69        }
70
71        System.out.println(prefix_2010 + (isTail_2010 ? "\\-- " : "/-- ") + data);
72
73        if (left != null) {
74            left.print(prefix_2010 + (isTail_2010 ? " " : "| "), true);
75        }
76    }
77
78 }
79
80 }
```

1. Deklarasi Kelas dan Atribut

Kode mendefinisikan kelas yang merepresentasikan satu titik simpul (node) dalam *Binary Tree*. Kelas ini menyimpan tiga informasi utama: data untuk menampung angka integer, left sebagai referensi penghubung ke anak cabang sebelah kiri, dan right sebagai penghubung ke anak cabang sebelah kanan.

2. Konstruktor Kelas

Saat sebuah objek Node baru dibuat (diinstansiasi), konstruktor `Node_2511532010(int data)` akan dijalankan. Konstruktor ini menyimpan nilai inputan ke dalam data dan memastikan bahwa cabang left dan right dimulai dalam keadaan null (kosong/belum memiliki anak).

3. Pengaturan Cabang Kiri dan Kanan (Setter)

Metode `setLeft()` dan `setRight()` digunakan untuk menambahkan node anak ke node saat ini. Terdapat pengkondisian khusus (`if == null`) di dalamnya yang berarti node anak hanya bisa ditambahkan jika slot cabang tersebut masih kosong, mencegah tertumpahnya data node anak yang sudah ada.

4. Pembacaan Data dan Cabang (Getter)

Metode `getLeft()`, `getRight()`, dan `getData()` bertugas untuk mengambil dan mengembalikan nilai dari node beserta jalur cabang anak-anaknya. Metode `setData()` disediakan jika nilai integer di dalam node tersebut perlu diperbarui.

5. Penelusuran Preorder (`printPreorder`)

Sebuah metode rekursif untuk membaca seluruh isi tree dengan urutan Cetak Akar (Root) -> Telusuri Kiri (Left) -> Telusuri Kanan (Right). Data akan langsung dicetak saat pertama kali node dikunjungi sebelum masuk ke cabang yang lebih dalam.

6. Penelusuran Postorder (`printPostorder`)

Metode rekursif untuk membaca isi tree dengan urutan Telusuri Kiri (Left) -> Telusuri Kanan (Right) -> Cetak Akar (Root). Program akan turun terus ke bagian daun paling bawah terlebih dahulu, lalu mencetak nilai dari bawah ke atas.

7. Penelusuran Inorder (`printInorder`)

Metode rekursif yang menelusuri tree dengan urutan Telusuri Kiri (Left) -> Cetak Akar (Root) -> Telusuri Kanan (Right). Jika dimasukkan ke dalam *Binary*

Search Tree (BST), penelusuran ini akan secara otomatis mencetak angka secara urut dari terkecil hingga terbesar.

8. Pencetakan Visual di Terminal (print)

Kumpulan metode cetak ini dirancang untuk menampilkan struktur tree secara grafis menggunakan karakter teks (seperti garis vertikal |, garis bawah --, dan spasi). Program menggunakan logika rekursif untuk mengatur jarak margin dan memastikan cabang atas-bawah terlihat seperti pohon bercabang saat dijalankan di layar *console*.

2.2 BTree_2511532010

```
1 package pekan9_2511532010;
2
3 public class BTree_2511532010 {
4     private Node_2511532010 root_2010;
5     private Node_2511532010 currentNode_2010;
6
7     public BTree_2511532010() {
8         root_2010 = null;
9     }
10
11    public boolean search (int data_2010) {
12        return search(root_2010, data_2010);
13    }
14
15    private boolean search(Node_2511532010 node_2010, int data_2010) {
16        if (node_2010 == null) return false;
17
18        if (node_2010.getData()==data_2010)
19            return true;
20        if (node_2010.getLeft()!= null)
21            if (search(node_2010.getLeft(), data_2010))
22                return true;
23        if (node_2010.getRight()!=null)
24            if (search(node_2010.getRight(), data_2010))
25                return true;
26        return false;
27    }
28
29    public void printInorder() {
30        root_2010.printInorder(root_2010);
31    }
32
33    public void printPreOrder() {
34        root_2010.printPreorder(root_2010);
35    }
36
37    public void printPostOrder() {
38        root_2010.printPostorder(root_2010);
39    }
40
41    public Node_2511532010 getRoot() {
42        return root_2010;
43    }
44
45    public boolean isEmpty() {
46        return root_2010 == null;
47    }
48
49    int countNodes(Node_2511532010 node_2010) {
50        int count_2010 = 1;
51        if (node_2010 == null) {
52            return 0;
53        } else {
54            count_2010 += countNodes(node_2010.getLeft());
55            count_2010 += countNodes(node_2010.getRight());
56            return count_2010;
57        }
58    }
59
60    public void print() {
61        root_2010.print();
62    }
63
64    public Node_2511532010 getCurrent() {
65        return currentNode_2010;
66    }
67
68    public void setCurrent(Node_2511532010 node_2010) {
69        this.currentNode_2010 = node_2010;
70    }
71
72    public void setRoot(Node_2511532010 root_2010) {
73        this.root_2010 = root_2010;
74    }
75 }
```

1. Struktur Dasar dan Atribut

Kelas ini bertindak sebagai pengelola utama pohon biner. Terdapat dua atribut privat: `root_2010` yang berfungsi sebagai akar (titik puncak tertinggi pohon), dan `currentNode_2010` yang bertindak sebagai *pointer* atau penunjuk ke simpul tertentu yang sedang aktif dikelola saat program berjalan.

2. Konstruktor Kelas

Ketika objek pohon (BTree) ini dibuat pertama kali di metode *main*, konstruktor `BTree_2511532010()` akan dijalankan untuk memastikan akar (`root_2010`) bernilai null. Ini berarti pohon biner diinisialisasi dalam keadaan kosong tanpa simpul apa pun.

3. Pencarian Data (Search)

Proses pencarian menggunakan teknik *overloading* (dua metode dengan nama sama namun parameter berbeda). Metode `search(int data_2010)` berfungsi sebagai pembuka jalan yang otomatis memulai pencarian dari akar. Kemudian, metode privat `search(Node_2511532010, int)` melakukan pencarian rekursif secara mendalam ke seluruh cabang kiri dan kanan. Jika nilai yang dicari cocok dengan data di suatu simpul, program mengembalikan `true`. Jika sampai ujung daun tidak ditemukan, ia mengembalikan `false`.

4. Penghitung Jumlah Simpul (countNodes)

Ini adalah fungsi rekursif yang bertugas menghitung berapa banyak total simpul (node) yang ada di dalam pohon. Logika matematikanya sederhana: jika node tidak kosong, hitung nilai node tersebut sebagai 1, lalu tambahkan dengan total node di seluruh cabang kirinya, ditambah total node di seluruh cabang kanannya. Jika node kosong, nilainya 0.

5. Pemanggilan Penelusuran (Traversal Wrappers)

Metode `printInorder()`, `printPreOrder()`, dan `printPostOrder()` bertindak sebagai tombol pemicu. Karena logika cara menelusuri pohon sudah ditulis di dalam kelas `Node_2511532010`, kelas ini hanya bertugas memerintahkan agar penelusuran tersebut dimulai dari posisi `root_2010`.

6. Pengecekan Pohon Kosong (isEmpty)

Fungsi validasi singkat yang akan mengembalikan status true jika root_2010 masih berisi null (pohon belum memiliki data sama sekali).

7. Pengambilan dan Pengaturan Posisi (Getter & Setter)

Kumpulan metode pembantu seperti `getRoot()`, `setRoot()`, `getCurrent()`, dan `setCurrent()`. Fungsi-fungsi ini menjadi pintu akses yang aman bagi program utama (kelas *Main*) untuk menunjuk, membaca, atau mengganti siapa yang menjadi akar pohon dan simpul mana yang sedang dikerjakan.

8. Pemanggilan Cetak Visual (`print`)

Sama seperti penelusuran traversal, fungsi `print()` di sini hanyalah pemicu yang menyuruh `root_2010` untuk menggambar dirinya sendiri (dan seluruh cabangnya) ke layar terminal menggunakan metode grafis yang sudah ada di kelas `Node`.

2.3 BtreeDriver_2511532010

```
1 package pekan9_2511532010;
2
3 public class BtreeDriver_2511532010 {
4     public static void main (String[] args) {
5         // Membuat Pohon
6         BTree_2511532010 tree = new BTree_2511532010();
7         System.out.print("Jumlah Simpul awal pohon: ");
8         // PERBAIKAN: Ganti null menjadi tree.getRoot()
9         System.out.println(tree.countNodes(tree.getRoot()));
10
11         //Menambahkan simpul data 1
12         Node_2511532010 root = new Node_2511532010 (1);
13         // Menjadikan simpul 1 sebagai root
14         tree.setRoot (root);
15         System.out.println("Jumlah simpul jika hanya ada root");
16         // PERBAIKAN: Ganti null menjadi tree.getRoot()
17         System.out.println(tree.countNodes(tree.getRoot()));
18
19         Node_2511532010 node2 = new Node_2511532010 (2);
20         Node_2511532010 node3 = new Node_2511532010 (3);
21         Node_2511532010 node4 = new Node_2511532010 (4);
22         Node_2511532010 node5 = new Node_2511532010 (5);
23         Node_2511532010 node6 = new Node_2511532010 (6);
24         Node_2511532010 node7 = new Node_2511532010 (7);
25         Node_2511532010 node8 = new Node_2511532010 (8);
26         Node_2511532010 node9 = new Node_2511532010 (9);
27
28         root.setLeft(node2);
29         node2.setLeft(node4);
30         node2.setRight(node5);
31         node4.setRight(node8);
32         root.setRight(node3);
33         node3.setLeft(node6);
34         node3.setRight(node7);
35         node6.setLeft(node9);
36
37         //Set root
38         tree.setCurrent(tree.getRoot());
39         System.out.println("menampilkan simpul terakhir: ");
40         System.out.println(tree.getCurrent().getData());
41
42         System.out.println("Jumlah simpul setelah semua simpul ditambahkan: ");
43         // PERBAIKAN: Ganti null menjadi tree.getRoot()
44         System.out.println(tree.countNodes(tree.getRoot()));
45
46         System.out.println("InOrder: ");
47         tree.printInorder();
48         System.out.println("\nPreorder: ");
49         tree.printPreOrder();
50         System.out.println("\nPostorder: ");
51         tree.printPostOrder();
52         System.out.println("\nMenampilkan simpul dalam bentuk pohon");
53         tree.print();
54     }
55 }
```

1. Inisialisasi Pohon Awal

Program membuat objek pohon biner kosong bernama tree dan membuktikan bahwa jumlah simpul awal adalah 0 menggunakan metode countNodes.

2. Pembuatan dan Pengaturan Root

Program membuat simpul baru bernilai 1 dan menentukannya sebagai akar utama pohon melalui `tree.setRoot(root)`, lalu memverifikasi bahwa jumlah simpul sekarang bertambah menjadi 1.

3. Instansiasi Simpul Tambahan

Program menciptakan delapan objek simpul independen di dalam memori (`node2` hingga `node9`) yang berisi angka 2 sampai 9 sebelum dirakit ke dalam pohon utama.

4. Perakitan Ranting Pohon

Program merangkai simpul-simpul tersebut secara manual menggunakan metode `setLeft()` dan `setRight()` untuk membentuk hierarki cabang atas-bawah dan kiri-kanan.

5. Pengujian Pointer Current

Program memosisikan indikator penunjuk kembali menunjuk ke arah akar pohon dan mencetak nilainya untuk memastikan fitur pelacakan simpul berjalan dengan benar.

6. Validasi Total Simpul Keseluruhan

Program menghitung ulang seluruh simpul setelah semuanya dirakit dan terhubung, yang akan membuktikan bahwa totalnya kini berjumlah 9 simpul.

7. Pengujian Penelusuran InOrder

Program mengeksekusi algoritma pembacaan dari bagian terdalam cabang kiri ke kanan yang akan menghasilkan urutan angka 4, 8, 2, 5, 1, 9, 6, 3, 7.

8. Pengujian Penelusuran PreOrder

Program mengeksekusi algoritma pembacaan dari akar turun ke dahan paling bawah yang akan menghasilkan urutan angka 1, 2, 4, 8, 5, 3, 6, 9, 7.

9. Pengujian Penelusuran PostOrder

Program mengeksekusi algoritma pembacaan dari ujung daun paling bawah naik menuju akar yang akan menghasilkan urutan angka 8, 4, 5, 2, 9, 6, 7, 3, 1.

10. Penggambaran Visual Pohon

HASIL :

13

2.4 GraphTransversal_2511532010

```
1 package pekan9_2511532010;
2
3 import java.util.*;
4 public class GraphTransversal_2511532010 {
5     private Map<String, List<String>> graph_2010 = new HashMap<>();
6
7     // Menambahkan edge (graf tak berarah)
8     public void addEdge (String node1_2010, String node2_2010) {
9         graph_2010.putIfAbsent(node1_2010, new ArrayList<>());
10        graph_2010.putIfAbsent(node2_2010, new ArrayList<>());
11        graph_2010.get(node1_2010).add(node2_2010);
12        graph_2010.get(node2_2010).add(node1_2010);
13    }
14
15    // Menampilkan graf awal
16    public void printGraph() {
17        System.out.println("Graf Awal (Adjacency List): ");
18        for (String node_2010 : graph_2010.keySet()) {
19            System.out.print(node_2010 + " -> ");
20            List<String> neighbors_2010 = graph_2010.get(node_2010);
21            System.out.println(String.join(", ", neighbors_2010));
22        }
23        System.out.println();
24    }
25
26    // DFS rekursif
27    public void dfs (String start_2010) {
28        Set<String> visited_2010 = new HashSet<>();
29        System.out.println("Penelusuran DFS: ");
30        dfsHelper(start_2010, visited_2010);
31        System.out.println();
32    }
33
34    private void dfsHelper(String current_2010, Set<String> visited_2010) {
35        if (visited_2010.contains(current_2010)) return;
36        visited_2010.add(current_2010);
37        System.out.print(current_2010 + " ");
38        for (String neighbor_2010 : graph_2010.getOrDefault(current_2010, new ArrayList<>())) {
39            dfsHelper(neighbor_2010, visited_2010);
40        }
41    }
42
43    // BFS iteratif
44    public void bfs (String start_2010) {
45        Set<String> visited_2010 = new HashSet<>();
46        Queue<String> queue_2010 = new LinkedList<>();
47        queue_2010.add(start_2010);
48        visited_2010.add(start_2010);
49        System.out.println("Penelusuran BFS:");
50        while (!queue_2010.isEmpty()) {
51            String current_2010 = queue_2010.poll();
52            System.out.print(current_2010 + " ");
53            for (String neighbor_2010 : graph_2010.getOrDefault(current_2010, new ArrayList<>())) {
54                if (!visited_2010.contains(neighbor_2010)) {
55                    queue_2010.add(neighbor_2010);
56                    visited_2010.add(neighbor_2010);
57                }
58            }
59        }
60        System.out.println();
61    }
62
63    // Main
64    public static void main (String[] args) {
65        GraphTransversal_2511532010 graph_obj_2010 = new GraphTransversal_2511532010();
66
67        // Contoh graf: A-B, A-C, B-D, B-E
68        graph_obj_2010.addEdge("A", "B");
69        graph_obj_2010.addEdge("A", "C");
70        graph_obj_2010.addEdge("B", "D");
71        graph_obj_2010.addEdge("B", "E");
72
73        // Cetak graf awal
74        graph_obj_2010.printGraph();
75
76        // Lakukan penelusuran
77        graph_obj_2010.dfs("A");
78        graph_obj_2010.bfs("A");
79    }
80 }
```

1. Inisialisasi Struktur Graf

Program mendeklarasikan struktur data *Map* (sebagai *Adjacency List*) untuk memetakan nama setiap simpul dengan daftar simpul tetangga yang terhubung dengannya.

2. Pembuatan Jalur Tak Berarah

Program mendefinisikan metode `addEdge` untuk membuat garis hubung bolak-balik antarsimpul, memastikan bahwa jika simpul 1 terhubung ke simpul 2, maka simpul 2 juga terhubung ke simpul 1.

3. Pencetakan Graf Awal

Program menggunakan metode `printGraph` untuk mengulang iterasi pembacaan seluruh isi *Map* dan menampilkan daftar ketetanggaan tiap simpul secara teks di terminal.

4. Inisialisasi Depth-First Search (DFS)

Program menyiapkan algoritma pencarian mendalam yang memanfaatkan struktur data *Set* untuk menyimpan riwayat simpul yang sudah dikunjungi agar tidak terjadi perulangan tanpa akhir (*infinite loop*).

5. Mekanisme Penelusuran DFS

Program mengeksekusi metode rekursif `dfsHelper` yang akan terus menyelusuri satu jalur lurus secara mendalam hingga ke ujung cabang sebelum mundur untuk memeriksa jalur tetangga lainnya.

6. Inisialisasi Breadth-First Search (BFS)

Program menyiapkan algoritma pencarian melebar yang mengkombinasikan struktur data *Set* untuk pelacakan kunjungan dan *Queue* (antrean) untuk mengatur jadwal eksekusi simpul berikutnya.

7. Mekanisme Penelusuran BFS

Program mengeksekusi perulangan berdasar antrean yang akan menelusuri seluruh simpul tetangga terdekat pada lapis pertama secara bergantian sebelum turun ke lapis simpul yang lebih jauh.

8. Perakitan Data Graf Utama

Program di dalam metode utama menginstansiasi objek graf lalu menambahkan garis hubung satu per satu untuk membangun relasi: A terhubung ke B dan C, serta B terhubung ke D dan E.

9. Penampilan Hasil Rakitan

Program memanggil fungsi pencetakan graf untuk memvalidasi secara visual bahwa titik simpul dan garis hubung dari proses perakitan sebelumnya telah tersimpan dengan benar di memori.

10. Pengujian DFS dan BFS

Program menjalankan kedua algoritma penelusuran yang dimulai dari titik awal "A", di mana DFS akan menghasilkan pola vertikal (A, B, D, E, C) sedangkan BFS menghasilkan pola horizontal/melebar (A, B, C, D, E).

HASIL :

```
Graf Awal (Adjacency List):  
A -> B, C  
B -> A, D, E  
C -> A  
D -> B  
E -> B  
  
Penelusuran DFS:  
A B D E C  
Penelusuran BFS:  
A B C D E
```

BAB III KESIMPULAN

3.1 Ringkasan Hasil Praktikum

Kesimpulan dari pelaksanaan praktikum Struktur Data ini adalah pemahaman komprehensif terkait pentingnya efisiensi dalam pengelolaan dan pengorganisasian data berbentuk linear. Melalui implementasi algoritma *Shell Sort*, *Quick Sort*, dan *Merge Sort* pada studi kasus *playlist* lagu, mahasiswa terbukti mampu menerjemahkan teori pengurutan ke dalam program yang fungsional. Mahasiswa tidak hanya dituntut untuk menyusun baris kode yang patuh terhadap aturan sintaksis dan penamaan atribut, tetapi juga memahami logika fundamental di balik pembagian kelompok data, pemilihan *pivot*, hingga proses penggabungan berulang. Hal ini memberikan wawasan teknis yang nyata bahwa setiap algoritma memiliki mekanisme unik dan tingkat efisiensi ruang serta waktu yang berbeda ketika menangani set data.

Selain manipulasi memori berformat linear, praktikum ini juga berhasil membekali mahasiswa dengan konsep representasi data non-linear yang jauh lebih kompleks melalui pemodelan *Binary Tree* dan *Graph*. Pembuatan kelas terstruktur untuk mengelola hierarki simpul (*node*), identifikasi akar utama (*root*), dan perakitan garis hubung (*edge*) memberikan gambaran arsitektur bagaimana data saling bertautan di dalam memori komputer. Pemahaman terhadap metode penelusuran hierarkis seperti *InOrder*, *PreOrder*, dan *PostOrder*, serta teknik pencarian jaringan berbasis kedalaman (*DFS*) maupun kelebaran (*BFS*), sangat mempertajam daya analisis algoritmik mahasiswa. Secara keseluruhan, rangkaian tugas praktikum ini berhasil menyatukan teori struktur data konseptual menjadi keahlian *problem-solving* praktis yang esensial bagi pengembangan perangkat lunak berskala besar.

DAFTAR PUSTAKA

- [1] Oracle, *The Java® Tutorial — Primitive Data Types*. [Online]. Available: <https://docs.oracle.com/javase/tutorial/java/nutsandbolts/datatypes.html>
- [2] GeeksforGeeks, *Java Data Types – GeeksforGeeks*. [Online]. Available: <https://www.geeksforgeeks.org/data-types-in-java>

TUGAS

1. PetaRelKereta

```

1 package pekan9_2511532010;
2
3 import javax.swing.*;
4 import java.awt.*;
5 import java.awt.event.ActionEvent;
6 import java.awt.event.ActionListener;
7 import java.util.*;
8
9
10 public class PetaRelKereta_2511532010 extends JFrame {
11
12     // Semua variabel menggunakan akhiran _2010 sesuai aturan
13     private Map<String, List<String>> graph_2010;
14     private Map<String, Point> koordinat_2010;
15     private List<String> nodes_2010;
16
17     private JComboBox<String> cbAwal_2010;
18     private JComboBox<String> cbTujuan_2010;
19     private JTextArea txtHasil_2010;
20     private PanelVisual_2010 panelGraph_2010;
21
22     private List<String> path_2010;
23     private List<String> visitedNodes_2010;
24     private int jumlahEksplorasi_2010;
25
26     public PetaRelKereta_2511532010() {
27         // Inisialisasi variabel koleksi
28         graph_2010 = new LinkedHashMap<>();
29         koordinat_2010 = new HashMap<>();
30         nodes_2010 = new ArrayList<>();
31         path_2010 = new ArrayList<>();
32         visitedNodes_2010 = new ArrayList<>();
33         jumlahEksplorasi_2010 = 0;
34
35         // Membanun data graf (10 Node, 15 Edge)
36         inisialisasiData_2010();
37
38         // Pengaturan JFrame
39         setTitle("Pencarian Jalur Kereta Api (BFS & DFS) - 2511532010");
40         setSize(850, 650);
41         setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
42         setLayout(new BorderLayout(10, 10));
43         setLocationRelativeTo(null);
44
45         // --- PANEL ATAS (Kontrol Input) ---
46         JPanel panelKontrol_2010 = new JPanel(new FlowLayout(FlowLayout.CENTER, 15, 15));
47         panelKontrol_2010.setBackground(new Color(30, 60, 100)); // Warna biru gelap
48
49         JLabel lblAwal_2010 = new JLabel("Stasiun Awal: ");
50         lblAwal_2010.setForeground(Color.WHITE);
51         cbAwal_2010 = new JComboBox<>(nodes_2010.toArray(new String[0]));
52
53         JLabel lblTujuan_2010 = new JLabel("Stasiun Tujuan: ");
54         lblTujuan_2010.setForeground(Color.WHITE);
55         cbTujuan_2010 = new JComboBox<>(nodes_2010.toArray(new String[0]));
56
57         JButton btnBfs_2010 = new JButton("[ BFS ]");
58         btnBfs_2010.setBackground(new Color(100, 200, 100)); // Hijau
59
60         JButton btnDfs_2010 = new JButton("[ DFS ]");
61         btnDfs_2010.setBackground(new Color(250, 180, 50)); // Oranye
62
63         JButton btnReset_2010 = new JButton("[ RESET ]");
64         btnReset_2010.setBackground(new Color(220, 80, 80)); // Merah
65
66         panelKontrol_2010.add(lblAwal_2010);
67         panelKontrol_2010.add(cbAwal_2010);
68         panelKontrol_2010.add(lblTujuan_2010);
69         panelKontrol_2010.add(cbTujuan_2010);
70         panelKontrol_2010.add(btnBfs_2010);
71         panelKontrol_2010.add(btnDfs_2010);
72         panelKontrol_2010.add(btnReset_2010);
73
74         // --- PANEL TENGAH (Visualisasi Graph) ---
75         panelGraph_2010 = new PanelVisual_2010();
76         panelGraph_2010.setBorder(BorderFactory.createTitledBorder("VISUALISASI GRAPH REL KERETA"));
77         panelGraph_2010.setBackground(new Color(240, 240, 245));
78
79         // --- PANEL BAWAH (Hasil Pencarian) ---
80         JPanel panelBawah_2010 = new JPanel(new BorderLayout());
81         panelBawah_2010.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
82         txtHasil_2010 = new JTextArea(6, 50);
83         txtHasil_2010.setEditable(false);
84         txtHasil_2010.setFont(new Font("Monospaced", Font.BOLD, 14));
85         txtHasil_2010.setText("Hasil Pencarian :\nJalur :\nNode Dikunjungi :\nJumlah Node Dikunjungi : 0");
86         panelBawah_2010.add(new JScrollPane(txtHasil_2010), BorderLayout.CENTER);
87
88         // Menambahkan panel ke Frame
89         add(panelKontrol_2010, BorderLayout.NORTH);
90         add(panelGraph_2010, BorderLayout.CENTER);
91         add(panelBawah_2010, BorderLayout.SOUTH);
92
93         // --- ACTION LISTENERS ---
94         btnBfs_2010.addActionListener(e -> BFS());
95         btnDfs_2010.addActionListener(e -> DFS());
96         btnReset_2010.addActionListener(e -> resetGraph());
97     }
98 }

```

```

99 private void inisialisasiData_2010() {
100     // 1. Menambahkan 10 Node (Stasiun) beserta koordinat untuk GUI
101     tambahNode_2010("Jakarta", 300, 50);
102     tambahNode_2010("Tangerang", 100, 120);
103     tambahNode_2010("Bekasi", 500, 120);
104     tambahNode_2010("Depok", 300, 150);
105     tambahNode_2010("Bogor", 300, 280);
106     tambahNode_2010("Cikarang", 650, 120);
107     tambahNode_2010("Karawang", 650, 220);
108     tambahNode_2010("Purwakarta", 500, 280);
109     tambahNode_2010("Cimahi", 400, 400);
110     tambahNode_2010("Bandung", 600, 400);
111
112     // 2. Menambahkan 15 Edge (Jalur Kereta)
113     tambahEdge_2010("Jakarta", "Tangerang");
114     tambahEdge_2010("Jakarta", "Bekasi");
115     tambahEdge_2010("Jakarta", "Depok");
116     tambahEdge_2010("Tangerang", "Depok");
117     tambahEdge_2010("Bekasi", "Depok");
118
119     tambahEdge_2010("Bekasi", "Cikarang");
120     tambahEdge_2010("Depok", "Bogor");
121     tambahEdge_2010("Bogor", "Purwakarta");
122     tambahEdge_2010("Cikarang", "Karawang");
123     tambahEdge_2010("Karawang", "Purwakarta");
124
125     tambahEdge_2010("Purwakarta", "Cimahi");
126     tambahEdge_2010("Purwakarta", "Bandung");
127     tambahEdge_2010("Cimahi", "Bandung");
128     tambahEdge_2010("Cikarang", "Purwakarta");
129     tambahEdge_2010("Karawang", "Bandung");
130 }
131
132 private void tambahNode_2010(String nama_2010, int x_2010, int y_2010) {
133     nodes_2010.add(nama_2010);
134     graph_2010.put(nama_2010, new ArrayList<>());
135     koordinat_2010.put(nama_2010, new Point(x_2010, y_2010));
136 }
137
138 private void tambahEdge_2010(String asal_2010, String tujuan_2010) {
139     graph_2010.get(asal_2010).add(tujuan_2010);
140     graph_2010.get(tujuan_2010).add(asal_2010); // Undirected graph
141 }
142
143 // --- METHOD WAJIB 1: BFS() ---
144 public void BFS() {
145     String start_2010 = (String) cbAwal_2010.getSelectedItem();
146     String goal_2010 = (String) cbTujuan_2010.getSelectedItem();
147
148     Queue<String> queue_2010 = new LinkedList<>();
149     Map<String, String> parent_2010 = new HashMap<>();
150
151     resetDataPencarian_2010();
152
153     queue_2010.add(start_2010);
154     visitedNodes_2010.add(start_2010);
155
156     boolean found_2010 = false;
157
158     while (!queue_2010.isEmpty()) {
159         String curr_2010 = queue_2010.poll();
160         jumlahEksplorasi_2010++;
161
162         if (curr_2010.equals(goal_2010)) {
163             found_2010 = true;
164             break;
165         }
166
167         for (String neighbor_2010 : graph_2010.get(curr_2010)) {
168             if (!visitedNodes_2010.contains(neighbor_2010)) {
169                 visitedNodes_2010.add(neighbor_2010);
170                 parent_2010.put(neighbor_2010, curr_2010);
171                 queue_2010.add(neighbor_2010);
172             }
173         }
174     }
175
176     if (found_2010) susunJalur_2010(parent_2010, start_2010, goal_2010);
177     displayPath();
178     displayGraph();
179 }
180
181 // --- METHOD WAJIB 2: DFS() ---
182 public void DFS() {
183     String start_2010 = (String) cbAwal_2010.getSelectedItem();
184     String goal_2010 = (String) cbTujuan_2010.getSelectedItem();
185
186     resetDataPencarian_2010();
187     Map<String, String> parent_2010 = new HashMap<>();
188
189     boolean found_2010 = dfsRekursif_2010(start_2010, goal_2010, parent_2010);
190
191     if (found_2010) susunJalur_2010(parent_2010, start_2010, goal_2010);
192     displayPath();
193     displayGraph();
194 }
195
196 private boolean dfsRekursif_2010(String curr_2010, String goal_2010, Map<String, String> parent_2010) {
197     visitedNodes_2010.add(curr_2010);

```

```

196 private boolean dfsRekursif_2010(String curr_2010, String goal_2010, Map<String, String> parent_2010) {
197     visitedNodes_2010.add(curr_2010);
198     jumlahEksplorasi_2010++;
199
200     if (curr_2010.equals(goal_2010)) {
201         return true;
202     }
203
204     for (String neighbor_2010 : graph_2010.get(curr_2010)) {
205         if (!visitedNodes_2010.contains(neighbor_2010)) {
206             parent_2010.put(neighbor_2010, curr_2010);
207             if (dfsRekursif_2010(neighbor_2010, goal_2010, parent_2010)) {
208                 return true;
209             }
210         }
211     }
212     return false;
213 }
214
215 // Helper untuk merangkai jalur setelah target ditemukan
216 private void susunJalur_2010(Map<String, String> parent_2010, String start_2010, String goal_2010) {
217     String curr_2010 = goal_2010;
218     while (curr_2010 != null) {
219         path_2010.add(0, curr_2010); // Insert di awal
220         curr_2010 = parent_2010.get(curr_2010);
221     }
222 }
223
224 // --- METHOD WAJIB 3: displayGraph() ---
225 public void displayGraph() {
226     panelGraph_2010.repaint(); // Memperbarui UI untuk mewarnai node
227 }
228
229 // --- METHOD WAJIB 4: displayPath() ---
230 public void displayPath() {
231     StringBuilder sb_2010 = new StringBuilder();
232     sb_2010.append("Hasil Pencarian :\n");
233
234     sb_2010.append("Jalur : ");
235     if (path_2010.isEmpty()) {
236         sb_2010.append("Jalur tidak ditemukan!\n");
237     } else {
238         sb_2010.append(String.join(" -> ", path_2010)).append("\n");
239     }
240
241     sb_2010.append("Node Dikunjungi : ").append(visitedNodes_2010.toString()).append("\n");
242     sb_2010.append("Jumlah Node Dikunjungi : ").append(jumlahEksplorasi_2010);
243
244     txtHasil_2010.setText(sb_2010.toString());
245 }
246
247 // --- METHOD WAJIB 5: resetGraph() ---
248 public void resetGraph() {
249     cbAwal_2010.setSelectedIndex(0);
250     cbTujuan_2010.setSelectedIndex(0);
251     resetDataPencarian_2010();
252     txtHasil_2010.setText("Hasil Pencarian :\nJalur :\nNode Dikunjungi :\nJumlah Node Dikunjungi : 0");
253     displayGraph();
254 }
255
256 private void resetDataPencarian_2010() {
257     visitedNodes_2010.clear();
258     path_2010.clear();
259     jumlahEksplorasi_2010 = 0;
260 }
261
262 // --- INNER CLASS UNTUK MENGGAMBAR GRAPH (Visualisasi Visual & Warna) ---
263 private class PanelVisual_2010 extends JPanel {
264     @Override
265     protected void paintComponent(Graphics g) {
266         super.paintComponent(g);
267         Graphics2D g2d_2010 = (Graphics2D) g;
268         g2d_2010.setRenderingHint(RenderingHints.KEY_ANTIALIASING, RenderingHints.VALUE_ANTIALIAS_ON);
269
270         // 1. Gambar Garis (Edge)
271         g2d_2010.setStroke(new BasicStroke(2));
272         g2d_2010.setColor(Color.GRAY);
273
274         // Track edges drawn untuk menghindari gambar dua kali
275         Set<String> drawnEdges_2010 = new HashSet<>();
276
277         for (String asal_2010 : graph_2010.keySet()) {
278             Point p1_2010 = koordinat_2010.get(asal_2010);
279             for (String tujuan_2010 : graph_2010.get(asal_2010)) {
280                 String edgeId1_2010 = asal_2010 + "-" + tujuan_2010;
281                 String edgeId2_2010 = tujuan_2010 + "-" + asal_2010;
282
283                 if (!drawnEdges_2010.contains(edgeId1_2010) && !drawnEdges_2010.contains(edgeId2_2010)) {
284                     Point p2_2010 = koordinat_2010.get(tujuan_2010);
285
286                     // Cek apakah garis ini adalah rute terpilih (jalur akhir)
287                     if (path_2010.contains(asal_2010) && path_2010.contains(tujuan_2010) &&
288                         Math.abs(path_2010.indexOf(asal_2010) - path_2010.indexOf(tujuan_2010)) == 1) {
289                         g2d_2010.setColor(new Color(50, 200, 50)); // Hijau untuk jalur
290                         g2d_2010.setStroke(new BasicStroke(4));
291                     } else {
292                         g2d_2010.setColor(Color.LIGHT_GRAY);
293                         g2d_2010.setStroke(new BasicStroke(2));
294                     }

```



```

295
296         g2d_2010.drawLine(p1_2010.x, p1_2010.y, p2_2010.x, p2_2010.y);
297         drawnEdges_2010.add(edgeId1_2010);
298     }
299 }
300
301 // 2. Gambar Titik (Node/Stasiun)
302 for (String node_2010 : nodes_2010) {
303     Point p_2010 = koordinat_2010.get(node_2010);
304
305     // Pewarnaan Berdasarkan Status (Sesuai Svarat)
306     if (path_2010.contains(node_2010)) {
307         g2d_2010.setColor(new Color(50, 200, 50)); // Hijau (Jalur Akhir)
308     } else if (visitedNodes_2010.contains(node_2010)) {
309         g2d_2010.setColor(new Color(255, 180, 50)); // Oranye (Telah dieksplorasi)
310     } else {
311         g2d_2010.setColor(new Color(150, 150, 150)); // Abu-abu (Belum dikunjungi)
312     }
313
314     g2d_2010.fillOval(p_2010.x - 15, p_2010.y - 15, 30, 30);
315
316     // Border Hitam pada Node
317     g2d_2010.setColor(Color.BLACK);
318     g2d_2010.drawOval(p_2010.x - 15, p_2010.y - 15, 30, 30);
319
320     // Teks Nama Stasiun
321     g2d_2010.setFont(new Font("Arial", Font.BOLD, 12));
322     g2d_2010.drawString(node_2010, p_2010.x - 20, p_2010.y - 20);
323 }
324 }
325 }
326 }
327
328 public static void main(String[] args) {
329     SwingUtilities.invokeLater(() -> {
330         new PetaRelKereta_2511532010().setVisible(true);
331     });
332 }
333 }

```

PENJELASAN:

1. Deklarasi Class dan Pewarisan

Class `PetaRelKereta_2511532010` melakukan `extends JFrame` yang berarti class ini bertindak secara langsung sebagai jendela utama aplikasi desktop.

2. Adjacency List

Menggunakan `Map<String, List<String>>` sebagai cara menyimpan graph di mana setiap nama stasiun menyimpan daftar stasiun tetangga yang terhubung langsung.

3. Penyimpanan Koordinat

Menggunakan `Map<String, Point>` karena aplikasi visual membutuhkan posisi koordinat X dan Y untuk setiap stasiun agar bisa digambar dengan presisi di layar.

4. Pelacakan Status Pencarian

Terdapat `path_2010` untuk menyimpan daftar stasiun rute akhir, `visitedNodes_2010` untuk mencatat stasiun yang sudah diperiksa, dan `jumlahEksplorasi_2010` sebagai alat penghitung.

5. Konstruktor Tampilan

Konstruktor `PetaRelKereta_2511532010()` dipanggil saat program pertama kali dijalankan untuk merakit antarmuka (UI) yang dibagi menjadi 3 area menggunakan `BorderLayout`.

6. Panel Kontrol Atas

Area ini diatur untuk meletakkan *Dropdown* pemilihan stasiun awal dan tujuan, beserta tiga tombol eksekusi utama yaitu BFS, DFS, dan RESET.

7. Panel Visual Tengah

Menggunakan class kustom `PanelVisual_2010` untuk bertindak sebagai kanvas kosong tempat gambar rute dan stasiun ditampilkan secara visual.

8. Panel Hasil Bawah

Berisi area teks (`JTextArea`) yang dapat digulir untuk menampilkan hasil keluaran berupa teks jalur, urutan stasiun dikunjungi, dan total node dieksplorasi.

9. Inisialisasi Data

Method `inisialisasiData_2010()` dijalankan otomatis di awal program untuk memasukkan data tiruan berupa 10 Node dan 15 Edge ke dalam memori.

10. Pembuatan Titik Stasiun

Method `tambahNode_2010` bertugas memasukkan nama stasiun, menyiapkan list kosong di graph, dan mendaftarkan titik koordinat visualnya.

11. Pembuatan Garis Rel

Method `tambahEdge_2010` menghubungkan dua stasiun secara bolak-balik karena ini adalah graf tak berarah (*undirected graph*).

12. Algoritma BFS

Saat tombol BFS diklik, program menggunakan struktur data Antrean (Queue) untuk bekerja secara melebar, memeriksa semua stasiun tetangga terdekat secara merata.

13. Pelacakan Parent BFS

Program menggunakan struktur `Map<String, String>` untuk mencatat dari stasiun mana mesin sampai di stasiun saat ini, agar bisa melacak jalan pulang nanti.

14. Looping Pencarian BFS

Proses akan terus berjalan selama antrian tidak kosong, berhenti seketika jika tujuan ditemukan, atau terus memasukkan tetangga baru ke antrian jika belum dikunjungi.

15. Algoritma DFS

Berbeda dengan BFS, tombol DFS memicu metode *Rekursif* (fungsi yang memanggil dirinya sendiri) untuk berjalan sejauh mungkin mengikuti satu cabang rute.

16. Proses Menyelam DFS

Algoritma ini akan terus menyelam ke tetangga pertama yang belum dikunjungi sampai menemukan jalan buntu di ujung rute.

17. Sistem Backtrack DFS

Jika bertemu jalan buntu, program akan mundur otomatis ke stasiun sebelumnya untuk mencoba rute persimpangan lain yang belum dilewati.

18. Rekonstruksi Jalur

Jika tujuan berhasil ditemukan oleh salah satu algoritma, method `susunJalur_2010()` dipanggil untuk merangkai rute secara utuh dari awal sampai akhir.

19. Penelusuran Mundur

Method rekonstruksi ini menelusuri mundur dari titik tujuan dengan melihat memori jejak riwayat hingga mencapai titik keberangkatan awal.

20. Penyisipan Rute Maju

Setiap stasiun yang ditarik mundur selalu disisipkan di posisi indeks paling depan agar hasil akhirnya tersusun maju dari Awal menuju Tujuan.

21. Pembaruan Teks Hasil

Method `displayPath()` mengambil data mentah dari hasil kalkulasi algoritma lalu merangkainya menjadi kalimat rapi untuk dicetak ke layar teks di panel bawah.

22. Visualisasi Gambar

Visualisasi grafis dilakukan menggunakan method `paintComponent` bawaan Java yang dilengkapi fitur *Anti-Aliasing* agar garis dan bentuk lingkaran tidak terlihat bergerigi.

23. Pewarnaan Garis Rel

Garis rel digambar tebal dan berwarna hijau jika dilalui rute penemuan, sedangkan sisa garis lainnya digambar tipis dan berwarna abu-abu.

24. Pewarnaan Titik Stasiun

Stasiun diwarnai hijau terang jika merupakan rute penemuan utama, oranye jika sempat dieksplorasi namun bukan jalan utama, dan abu-abu jika sama sekali tidak tersentuh.

25. Fungsi Reset Program

Method `resetGraph()` mengembalikan seluruh form ke setelan awal, menghapus memori jejak pencarian, dan membuat panel gambar kembali menjadi abu-abu bersih.

26. Entry Point Main

Program diawali menggunakan `SwingUtilities.invokeLater()` untuk memastikan elemen antarmuka berjalan di ruang memori yang aman agar aplikasi tidak macet saat mencari jalur.